

PROPLEDGER ARCHITECTURE SERIES — VOLUME 06 OF 10

Identity & Spring Security

Stateless JWT Architecture, RBAC & Cryptographic Verification

DOMAIN LEVEL:

Enterprise Real Estate SaaS

Comparable System: Yardi Voyager / RealPage

FRAMEWORKS:

Spring Boot 3.3 | PG 16 | React 19

PAGE BUDGET:

10 Pages Complete Reference

1. DOCUMENT EXECUTIVE ABSTRACT & SCOPE

This volume presents the zero-trust security and authentication architecture of PropLedge. Built with Spring Security 6.3 and Java 21, it provides an exhaustive code-level analysis of stateless JSON Web Tokens (RFC 7519), BCrypt password hashing, OncePerRequestFilter request interception, fine-grained Role-Based Access Control (RBAC), and enterprise attack mitigation strategies.

2. VOLUME SYLLABUS & CORE CAPABILITIES

Page	Chapter Title	Primary Technical Focus & Learning Outcome
02	Role-Based Access Control (RBAC)	Hierarchical privilege tiers from Super Admin to Resident
03	Stateless JWT vs Stateful Sessions	Eliminating sticky sessions and session database bottlenecks
04	RFC 7519 Token Anatomy	Header, claims payload, and HMAC-SHA256 signature mechanics
05	End-to-End Authentication Flow	From login credentials to SecurityContextHolder thread-binding
06	Deep-Dive: JwtTokenProvider	Token generation, claims parsing, and cryptographic verification
07	Deep-Dive: JwtAuthenticationFilter	OncePerRequestFilter interceptor extracting Bearer headers
08	Deep-Dive: SecurityConfig	Stateless SecurityFilterChain, CORS policy, and CSRF disabling
09	Attack Vectors & Mitigations	XSS vs CSRF, secret entropy, and token revocation strategies
10	Security & Auth Interview Q&A	Senior engineering interview scenarios, JWE, and refresh tokens

SENIOR STAFF ENGINEER REVIEWER GUIDANCE

This volume is engineered specifically for software engineering and database architecture interviews at Tier-1 real estate technology and enterprise SaaS corporations (Yardi, RealPage, AppFolio, CoStar). Every code sample reflects active, compilable implementations from the PropLedge production repository.

2. ROLE-BASED ACCESS CONTROL (RBAC) PRIVILEGE MATRIX

ENTERPRISE PERSONAS & LEAST PRIVILEGE SEGREGATION

1. The Enterprise Persona Hierarchy

Enterprise real estate software manages sensitive lease contracts, bank payouts, and tenant PII. PropLedge enforces strict segregation of duties across five role tiers:

Enterprise Role	Asset Mgmt	Tenant / Leases	Billing / Invoices	Audit Logs	System Config
ROLE_SUPER_ADMIN	Full Control	Full Control	Full Control	Full Access	Full Control
ROLE_PROPERTY_MANAGER	Create / Edit	Onboard / Sign	Generate / Send	Read Own	No Access
ROLE_ACCOUNTANT	Read Only	Read Only	Post / Settle	Read Ledger	No Access
ROLE_MAINTENANCE_Tech	Read Spaces	Read Work Tickets	Log Labor Costs	No Access	No Access
ROLE_TENANT	No Access	Read Own Lease	Pay Own Invoices	No Access	No Access

2. Method-Level Authorization via @PreAuthorize

PropLedge combines endpoint-level filter rules with method-level Spring-EL security annotations:

```
// CODE LISTING: JAVA 21 – METHOD-LEVEL AUTHORIZATION

// Method-level Security in Controller Layer
@DeleteMapping("/{id}")
@PreAuthorize("hasRole('SUPER_ADMIN')")
public ResponseEntity<ApiResponse<Void>> deleteProperty(@PathVariable Long id) {
    propertyService.deleteProperty(id);
    return ResponseEntity.ok(ApiResponse.success("Property deactivated", null));
}
```

3. STATELESS JWT VS STATEFUL SERVER-SIDE SESSIONS

HORIZONTAL SCALABILITY & ZERO-DATABASE SESSION BOTTLENECKS

1. The Architectural Breakdown of Stateful Sessions

In legacy session-based architectures (JSESSIONID stored in cookies): 1) Every API request hits a centralized Redis cluster to deserialize session state. If Redis experiences latency, the entire API stalls. 2) Load balancers require 'sticky sessions', preventing uniform traffic distribution.

2. The Stateless JWT Paradigm in PropLedger

In PropLedger, the server signs a cryptographically verifiable token containing the user's ID and granted roles. Any backend container can verify the token's validity in sub-millisecond time using pure mathematics, with zero database lookups:

Evaluation Metric	Stateful Server Sessions (Redis)	Stateless JWT (PropLedger)
Database I/O on Request	Requires 1 Redis/DB read per HTTP call	ZERO database reads (pure math verification)
Horizontal Scaling	Requires shared distributed session cache	Effortless (instances share only public/private key)
Cross-Domain / Mobile	Complex cookie domain & third-party blocks	Simple (passed in Authorization: Bearer header)
Token Revocation	Instant (delete key in Redis)	Requires short expiration or JTI blacklist cache

4. ANATOMY OF A JSON WEB TOKEN (RFC 7519)

HEADER, CLAIMS PAYLOAD & CRYPTOGRAPHIC SIGNATURE

1. Three Token Components

A JWT is a compact, URL-safe string containing three base64url-encoded parts separated by periods:

- 1. Header: Algorithm (HS256) and token type (JWT): '{"alg": "HS256", "typ": "JWT"}'.
- 2. Payload (Claims): Identity statements: subject ('manager@propledger.com'), roles (['ROLE_PROPERTY_MANAGER']), issuedAt, expiration.
- 3. Signature: Cryptographic hash guaranteeing tamper-proof integrity:

Signature = HMAC-SHA256(base64Url(Header) + '.' + base64Url(Payload), SecretKey)

2. Real Decoded Token Payload from PropLedger

// CODE LISTING: JSON – DECODED CLAIMS PAYLOAD

```
{
  "sub": "manager@propledger.com",
  "userId": 1,
  "fullName": "Enterprise Property Manager",
  "roles": [
    "ROLE_PROPERTY_MANAGER"
  ],
  "iat": 1726100000,
  "exp": 1726186400
}
```

SECURITY GOLDEN RULE

A JWT is signed, NOT encrypted! Anyone who intercepts the token can decode the claims. Never store sensitive secrets (passwords, social security numbers, credit card numbers) in a JWT payload.

5. END-TO-END AUTHENTICATION & AUTHORIZATION FLOW

SEQUENCE OF OPERATIONS: LOGIN TO SECURITYCONTEXTHOLDER

1. Sequence of Authentication Operations

- 1. Login Request: Browser POSTs { username, password } to /api/auth/login.
- 2. AuthenticationManager: Delegates to DaoAuthenticationProvider -> UserDetailsServiceImpl.
- 3. BCrypt Verification: Hashes candidate password with 12 rounds of salting and compares against users.password_hash.
- 4. Token Generation: JwtTokenProvider constructs signed JWT string and returns it in HTTP 200 payload.
- 5. Client Storage: React saves token in localStorage and React Context (AuthContext.tsx).
- 6. Authenticated Request: Axios interceptor attaches 'Authorization: Bearer <token>' to all subsequent API calls.
- 7. Filter Interception: JwtAuthenticationFilter extracts token, validates signature, builds Authentication object, and populates SecurityContextHolder.

2. Password Hashing: BCrypt Work Factor 12

PropLedger enforces BCrypt with work factor 12 ($2^{12} = 4,096$ iterations). This provides an optimal balance between server login latency (~200ms) and resistance against brute-force GPU attacks.

6. DEEP-DIVE: JWTTOKENPROVIDER IMPLEMENTATION

GENERATING, SIGNING & PARSING CRYPTOGRAPHIC TOKENS

1. JwtTokenProvider.java Source Listing

```
// CODE LISTING: JAVA 21 – JWTTOKENPROVIDER.JAVA
// propledger-backend/src/main/java/com/propledger/security/JwtTokenProvider.java
@Component
@Slf4j
public class JwtTokenProvider {
    @Value("${app.jwt.secret}") private String jwtSecret;
    @Value("${app.jwt.expiration-ms}") private long jwtExpirationMs;

    private SecretKey getSigningKey() {
        byte[] keyBytes = Decoders.BASE64.decode(jwtSecret);
        return Keys.hmacShaKeyFor(keyBytes);
    }

    public String generateToken(Authentication authentication) {
        UserDetails principal = (UserDetails) authentication.getPrincipal();
        Date now = new Date();
        Date expiry = new Date(now.getTime() + jwtExpirationMs);

        List<String> roles = principal.getAuthorities().stream()
```

7. DEEP-DIVE: JWTAUTHENTICATIONFILTER

ONCEPERREQUESTFILTER & SECURITYCONTEXT INJECTION

1. JwtAuthenticationFilter.java Source Listing

```
// CODE LISTING: JAVA 21 – JWTAUTHENTICATIONFILTER.JAVA
```

```
// propledger-backend/src/main/java/com/propledger/security/JwtAuthenticationFilter.java
@Component
@RequiredArgsConstructor
public class JwtAuthenticationFilter extends OncePerRequestFilter {
    private final JwtTokenProvider tokenProvider;
    private final UserDetailsService userDetailsService;

    @Override
    protected void doFilterInternal(HttpServletRequest request, HttpServletResponse response,
                                    FilterChain filterChain) throws ServletException, IOException {
        String jwt = getJwtFromRequest(request);

        if (StringUtils.hasText(jwt) && tokenProvider.validateToken(jwt)) {
            String username = tokenProvider.getUsernameFromToken(jwt);
            UserDetails userDetails = userDetailsService.loadUserByUsername(username);

            UsernamePasswordAuthenticationToken authentication =
                new UsernamePasswordAuthenticationToken(userDetails, null, userDetails.getAuthorities());
```

8. DEEP-DIVE: SECURITYCONFIG & FILTERCHAIN

SPRING SECURITY 6.3 DSL & STATELESS ARCHITECTURE

1. SecurityConfig.java Source Listing

```
// CODE LISTING: JAVA 21 – SECURITYCONFIG.JAVA
// propledger-backend/src/main/java/com/propledger/config/SecurityConfig.java
@Configuration
@EnableWebSecurity
@EnableMethodSecurity
@RequiredArgsConstructor
public class SecurityConfig {
    private final JwtAuthenticationFilter jwtAuthenticationFilter;

    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
        http
            .csrf(AbstractHttpConfigurer::disable) // Disabled: APIs are stateless Bearer auth
            .cors(Customizer.withDefaults())
            .sessionManagement(s -> s.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/api/auth/**", "/api/v1/auth/**").permitAll()
                .requestMatchers("/swagger-ui/**", "/v3/api-docs/**").permitAll()
                .requestMatchers(HttpMethod.GET, "/api/properties/**").authenticated()
            )
    }
}
```

9. ATTACK VECTORS & SECURITY MITIGATIONS

XSS, CSRF, TOKEN REVOCATION & SECRET ENTROPY

1. Security Analysis: XSS vs CSRF

Because Bearer tokens are stored in the browser and transmitted explicitly via the Authorization header, CSRF attacks are impossible (browsers never send custom headers automatically on cross-origin links).

To protect against XSS (Cross-Site Scripting), PropLedger implements: 1) Strict HTML input sanitization on all text fields. 2) Content Security Policy (CSP) headers. 3) Short token expiration windows.

2. Token Revocation Strategies Compared

Revocation Strategy	Mechanism	Latency Impact	Complexity
Short Expiration (Selected)	Tokens expire in 24 hours; refresh required	Zero latency overhead	Simple & Stateless
User token_version Counter	Increment counter in users table on password c...	Requires checking 1 cached in...	Low
Redis JTI Blacklist	Store revoked token IDs (JTI) in Redis with TTL	Requires 1 Redis check per re...	Medium (adds state)

10. SECURITY & AUTHENTICATION INTERVIEW Q&A

SENIOR SYSTEMS DESIGN & SECURITY ENGINEERING SCENARIOS

1. Real-World Security Interview Scenarios

Question 1: What is the difference between Access Tokens and Refresh Tokens?

Answer: Access tokens are short-lived (15–30 minutes) and sent on every API request. Refresh tokens are long-lived (7–30 days), stored securely (e.g. HttpOnly cookie), and used solely to request a new access token when the current one expires. This limits exposure if an access token is intercepted while maintaining a seamless user login experience.

Question 2: What is the difference between Symmetric (HS256) and Asymmetric (RS256) signing?

Answer: HS256 uses a shared private key for both signing and verification. RS256 uses an RSA keypair: the auth server signs with a private key, while downstream microservices verify with a public key without needing the private key. For PropLedge's unified Spring Boot cluster, HS256 provides optimal performance.

Question 3: How does Spring Security prevent user credentials from leaking into heap dumps?

Answer: Spring Security's AuthenticationManager clears plaintext password credentials from the Authentication object immediately after successful authentication (`eraseCredentialsAfterAuthentication = true`), ensuring sensitive plaintext strings do not persist in long-lived JVM heap memory.

SECURITY ARCHITECTURE KEY TAKEAWAY

Articulating why CSRF is safely disabled for stateless Bearer APIs and explaining `token_version` revocation proves high security maturity to interviewers.